

FACULDADE ENGENHEIRO SALVADOR ARENA

HENRIQUE ALVES FERREIRA – 081230015

GABRIEL MELO SANTOS – 081230044

MATHEUS DA SILVA SOUZA – 081230011

RAFAEL RUPPERT BARROCAL – 081230002

DOCUMENTAÇÃO TÉCNICA:

SIMULADOR DE ESCALONAMENTO DE TAREFAS

SÃO BERNARDO DO CAMPO

2026

SUMÁRIO

1 VISÃO GERAL.....	3
2 SEPARAÇÃO ENTRE POLÍTICA E MECANISMO	4
3 DIAGRAMA DE MÓDULOS	5
4 ESTRUTURA DE UMA TAREFA.....	6
5 INTERFACE DOS MÓDULOS.....	7
6 PARÂMETROS CONFIGURÁVEIS	16
7 FUNCIONAMENTO INTERNO.....	17
8 CONVENÇÕES DE SIMULAÇÃO	18
9 DEPENDÊNCIAS E AMBIENTE	19

1 VISÃO GERAL

O projeto presente consiste em um simulador de escalonamento de tarefas em um processador único em tempo discreto, na forma de uma aplicação desktop. A simulação visa reproduzir visualmente a lógica aplicada por sistemas operacionais reais para priorizar o processamento de diferentes tarefas. A aplicação é feita em Python e conta com interface gráfica.

O programa permite simular os seis algoritmos de escalonamento:

- FCFS: priorização por ordem de chegada;
- SJF: priorização por menor duração do processamento;
- SRTF: priorização por menor tempo restante de processamento, com preempção;
- Round Robin: alternância cíclica entre tarefas, segundo um dado quantum;
- Prioridade Cooperativa: priorização por maior valor de prioridade, sem preempção;
- Prioridade Preemptiva: priorização por maior valor de prioridade, com preempção.

É possível inserir tarefas com seus parâmetros e simular cada um dos algoritmos. A simulação retorna as métricas resultantes e produz um diagrama de Gantt, exibindo as tarefas processadas e suspensas ao longo do tempo.

Para a simulação de prioridade cooperativa, é possível definir um valor de alfa para simular o envelhecimento e prevenir inanição. Também pode-se definir uma seção crítica para as tarefas, onde um recurso é utilizado de forma exclusiva. Com prioridade preemptiva, é possível simular herança e teto de prioridade, que são tratativas para priorizar a utilização de recursos.

Na entrada de tarefas é possível, alternativamente à inserção manual, sortear uma dado número de tarefas, bem como exportar e importar um conjunto de tarefas de um arquivo json. Outro modo de simulação é em lotes, onde é possível gerar e um dado número de tarefas e simulá-las um dado número de vezes para cada algoritmo, apresentando as médias resultantes.

2 SEPARAÇÃO ENTRE POLÍTICA E MECANISMO

O ponto de entrada do mecanismo de simulação, comum a todos os algoritmos, é a função `simular` em `control/motor.py`. Ele recebe como parâmetros a lista de processos (`processos`), a política aplicada (`politica`), o tempo de troca de contexto (`ctx_time`), o valor de alfa para definir envelhecimento (`alfa`) e o protocolo de priorização de uso de recursos (`protocolo`). Na função `simular`, ocorre o direcionamento de qual variante de laço rodar, dependendo dos parâmetros da política aplicada.

A política, por sua vez, é modelada pelo dataclass `Politica`, no arquivo `control/politicas.py`. Seus atributos são a chave de seleção (`chave_selecao`) – callable que pode ser uma das várias funções de chave presentes na classe, que vai definir a ordenação dos processos em cada política – e as opções de preempção (`preemptivo`), uso de quantum (`usa_quantum`), envelhecimento (`suporta_envelhecimento`) e simulação de recursos (`suporta_recursos`). Esses parâmetros são o que define as tratativas que o motor vai tomar para o conjunto de tarefas.

A função `_simular_por_chave` é o laço comum utilizado por FCFS, SJF, SRTF e PRIOc. Com a chave de seleção da política, ele seleciona a próxima tarefa do conjunto. Se a política conta com preempção, ele executa uma unidade de tempo a cada iteração; se não a execução vai até o fim da tarefa. Se uma tarefa diferente for selecionada, ocorre a troca de contexto. O laço é agnóstico ao critério de seleção, e apenas aplica a chave callable da política.

Duas exceções se desviam desse laço comum: Round-Robin e PRIOp. Round-Robin é simulado por `_simular_round_robin`, pois usa critério de *first-in-first-out*, com alternância por quantum, e não é aplicável ao padrão do laço comum. PRIOp precisa filtrar candidatos suspensos e recalcular a prioridade a cada evento, por isso tem função separada: `_simular_prioridade_preemptiva_com_recursos`.

3 DIAGRAMA DE MÓDULOS

A figura 1 ilustra os módulos que compõem o simulador.

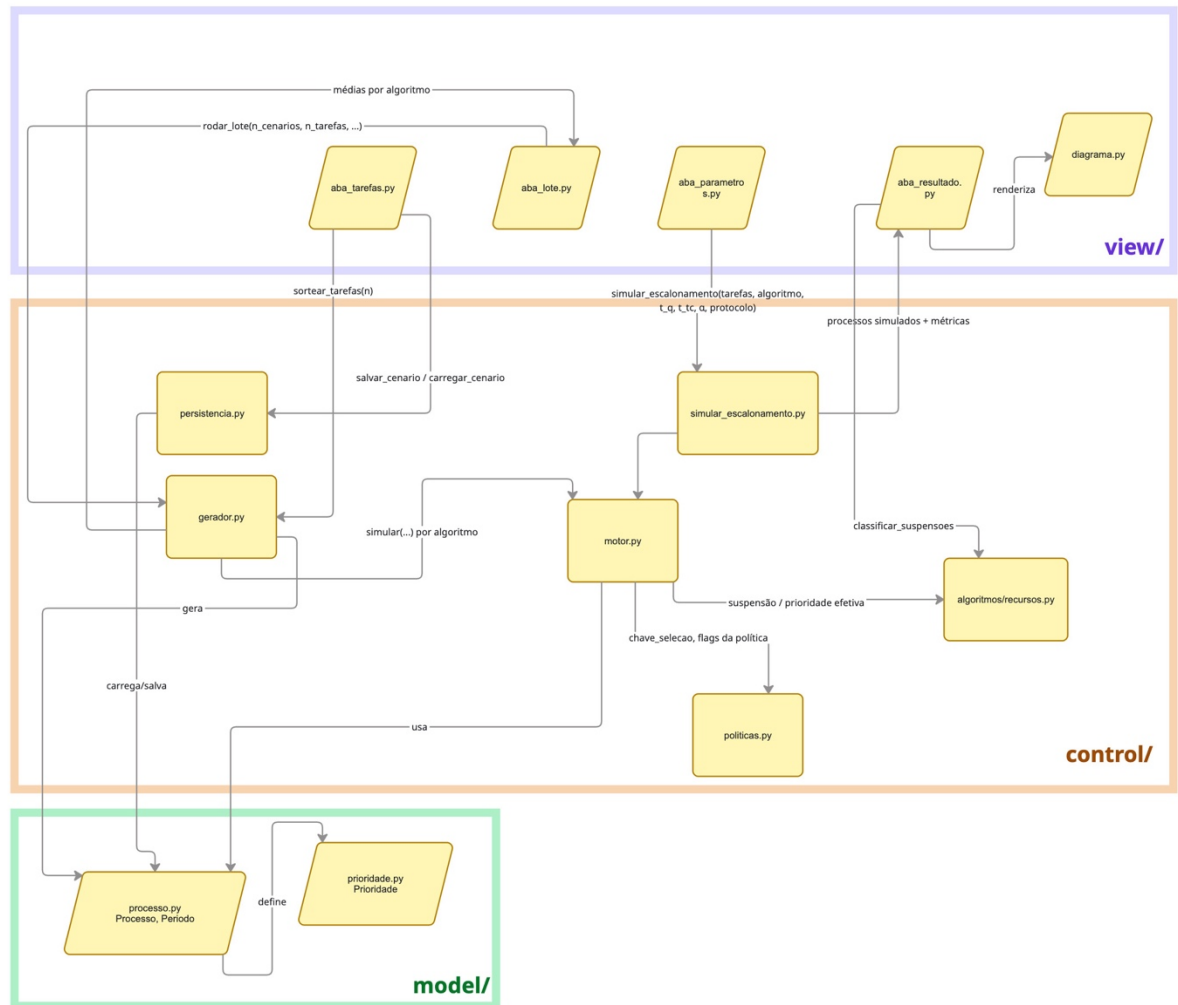


Figura 1 - diagrama de módulos

4 ESTRUTURA DE UMA TAREFA

A tabela 1 representa a estrutura que modela uma tarefa no simulador (`model/processo.py`). O período da tarefa, item da lista processamentos é representado na tabela 2.

Campo	Tipo	Função
<code>id</code>	<code>int</code>	Identificador
<code>chegada</code>	<code>int</code>	Instante de ingresso
<code>duracao</code>	<code>int</code>	Tempo de processamento total (t_p)
<code>prioridade</code>	Prioridade (numero: <code>int</code>)	Prioridade (crescente)
<code>secao_critica</code>	<code>SecaoCritica</code> <code>None</code>	<code>recurso (str)</code> , <code>inicio_execucao (int)</code> , unidades já executadas até obter o <code>recurso</code>), <code>duracao (int)</code> , por quantas unidades o mantém)
<code>tempo_restante</code>	<code>int</code>	Quanto falta excutar
<code>processamentos</code>	<code>list[Periodo]</code>	Histórico de períodos

Tabela 1 - estrutura da tarefa

Campo	Tipo	Função
<code>inicio</code>	<code>int</code>	Instante de início do período
<code>fim</code>	<code>int</code>	Instante de fim do período
<code>tipo</code>	<code>str</code>	“Execução”, “CTX” (troca de contexto) ou “Suspensa”.

Tabela 2 - estrutura do período

5 INTERFACE DOS MÓDULOS

`control/simular_escalonamento.py`

- `simular_escalonamento`: ponto de entrada público da simulação; valida a entrada e chama o motor.
 - Parâmetros:
 - `processos`: lista de tarefas para serem simuladas;
 - `algoritmo`: nome do algoritmo
 - `quantum_entry`: duração do quantum, se houver;
 - `ctx_time`: tempo de troca de contexto;
 - `alfa`: fator de envelhecimento;
 - `protocolo`: nome do protocolo de priorização de uso de recursos;
 - Retorna:
 - `media_execucao`: tempo médio de execução;
 - `media_espera`: tempo médio de espera;
 - `nome_processo`: nome do algoritmo de escalonamento;
 - `processos_simulados`: processos após a simulação;

`control/motor.py`

- `simular`: função única que o resto do sistema chama para simular um algoritmo; decide qual variante do laço usar (fila circular, seleção por chave, ou seleção por chave com suspensão por recurso) verificando as flags da política recebida.
 - Parâmetros:
 - `processos`: lista de tarefas a simular;
 - `politica`: objeto que declara o critério de escolha e as flags do algoritmo (se usa quantum, se suporta envelhecimento, se suporta recursos);
 - `quantum`: duração da fatia de tempo, usada só quando a política usa quantum;
 - `ctx_time`: custo da troca de contexto;
 - `alfa`: fator de envelhecimento, usado só quando a política suporta envelhecimento;

- protocolo: protocolo de correção de recurso (“herança”, “teto” ou nenhum), usado só quando a política suporta recursos;
- Retorna:
 - media_espera: tempo médio de espera entre as tarefas simuladas;
 - media_execucao: tempo médio de execução (turnaround);
 - nome: nome de exibição do algoritmo simulado;
- _simular_por_chave: laço comum a FCFS, SJF, SRTF, Prioridade Cooperativa e Prioridade Preemptiva sem recurso. A cada evento, monta o conjunto de tarefas prontas, entrega à chave de seleção da política escolhida, despacha a tarefa retornada por uma unidade de tempo (se preemptivo) ou até o fim (se não), e cobra a troca de contexto quando a tarefa despachada muda.
 - Parâmetros:
 - processos: lista de tarefas a simular;
 - chave_selecao: função que recebe uma tarefa e o instante atual e devolve o critério de comparação usado para escolher a próxima;
 - preemptivo: indica se a tarefa roda por uma unidade de tempo (verdadeiro) ou até concluir (falso);
 - ctx_time: custo da troca de contexto;
 - Retorna:
 - total_espera: soma dos tempos de espera de todas as tarefas;
 - total_execucao: soma dos tempos de execução de todas as tarefas;
- _simular_prioridade_preemptiva_com_recursos: variante do laço anterior usada pela Prioridade Preemptiva quando há tarefas com seção crítica declarada. A cada evento, identifica quais tarefas prontas estão suspensas e as remove do conjunto de candidatas; recalcula a prioridade efetiva de quem detém um recurso, conforme o protocolo ativo; e grava, ao final de cada suspensão, o período correspondente na própria tarefa.
 - Parâmetros:
 - processos: lista de tarefas a simular;
 - ctx_time: custo da troca de contexto;
 - protocolo: protocolo de correção ativo;
 - Retorna:

- `total_espera`: soma dos tempos de espera de todas as tarefas;
- `total_execucao`: soma dos tempos de execução de todas as tarefas;
- `_simular_round_robin`: laço específico do Round Robin, que usa uma fila circular para decidir a ordem de despacho. Move para a fila as tarefas já chegadas, retira a primeira, despacha por uma fatia de tempo e recoloca ao final da fila quem não concluiu.
 - Parâmetros:
 - `processos`: lista de tarefas a simular;
 - `quantum`: duração da fatia de tempo concedida a cada despacho;
 - `ctx_time`: custo da troca de contexto;
 - Retorna:
 - `total_espera`: soma dos tempos de espera de todas as tarefas;
 - `total_execucao`: soma dos tempos de execução de todas as tarefas;

`control/algoritmos/_base.py`

- `avancar_tempo_para_chegada`: adianta o relógio da simulação até o instante de ingresso de uma tarefa, caso o relógio esteja atrasado em relação a ela.
 - Parâmetros:
 - `processo`: tarefa cujo instante de ingresso serve de referência;
 - `tempo_atual`: instante atual do relógio da simulação;
 - Retorna:
 - o instante de ingresso da tarefa, se for posterior ao relógio atual; o próprio relógio atual, caso contrário;
- `chave_desempate`: critério de desempate usado por todas as políticas: menor instante de ingresso, depois menor identificador.
 - Parâmetros:
 - `processo`: tarefa a ser comparada;
 - `tempo_atual`: não utilizado, existe só para manter a mesma assinatura das demais chaves;
 - Retorna:
 - uma tupla (chegada, id) usada como critério de comparação;

- `selecionar_proximo`: dentre as tarefas já chegadas, escolhe a de menor chave; a chave é calculada com o instante atual, o que permite que ela dependa do tempo (no caso do envelhecimento).
 - Parâmetros:
 - `processos_ordenados`: lista de tarefas ainda pendentes;
 - `tempo_atual`: instante atual do relógio da simulação;
 - `chave`: função que recebe uma tarefa e o instante atual e devolve o critério de comparação;
 - Retorna:
 - a tarefa pronta de menor chave; se nenhuma tarefa tiver chegado ainda, a primeira da lista recebida;
- `trocar_contexto_padrao`: cobra o custo de uma troca de contexto sempre que a tarefa despachada é diferente da última que ocupou o processador ou no primeiro despacho. O custo é registrado como um período de troca de contexto na própria tarefa que está entrando.
 - Parâmetros:
 - `ultimo_processo`: última tarefa que ocupou o processador (ou nenhuma, se for o primeiro despacho);
 - `processo_atual`: tarefa prestes a ser despachada;
 - `tempo_atual`: instante atual do relógio da simulação;
 - `ctx_time`: custo da troca de contexto;
 - Retorna:
 - o instante atual, avançado pelo custo da troca, se ela ocorreu; o próprio instante atual, caso contrário;
- `despachar_com_quantum`: despacha uma tarefa sob Round Robin por uma fatia de tempo, descontando o custo da troca de contexto da própria fatia quando há troca.
 - Parâmetros:
 - `ultimo_processo`: última tarefa que ocupou o processador;
 - `processo_atual`: tarefa sendo despachada;
 - `tempo_atual`: instante atual do relógio da simulação;
 - `quantum`: duração da fatia concedida;
 - `ctx_time`: custo da troca de contexto;
 - Retorna:

- o instante atual, avançado pelo tempo efetivamente consumido (troca de contexto, se houve, mais a execução);

control/algoritmos/recursos.py

- `esta_na_secao_critica`: verifica se uma tarefa está, neste momento, dentro da própria seção.
 - Parâmetros:
 - `p`: tarefa a verificar;
 - Retorna:
 - verdadeiro se a tarefa declara seção crítica e já executou entre o início e o fim dela; falso caso contrário;
- `detentor_do_recurso`: procura, entre um conjunto de tarefas, qual delas está atualmente dentro da seção crítica de um determinado recurso.
 - Parâmetros:
 - `tarefas`: conjunto de tarefas onde procurar;
 - `recurso`: identificador do recurso;
 - Retorna:
 - a tarefa que detém o recurso no momento; nenhuma, se ele estiver livre;
- `bloqueada_por_recurso`: verifica se uma tarefa deveria ficar suspensa agora — ela está prestes a entrar na própria seção crítica, mas o recurso já está ocupado por outra tarefa.
 - Parâmetros:
 - `p`: tarefa a verificar;
 - `tarefas`: conjunto de tarefas onde procurar o detentor do recurso;
 - Retorna:
 - verdadeiro se a tarefa deve ficar suspensa; falso caso contrário;
- `calcular_tetos`: calcula, uma única vez por simulação, o teto de cada recurso.
 - Parâmetros:
 - `processos`: lista completa de tarefas da simulação;
 - Retorna:
 - um mapa de recurso para o valor do seu teto;
- `prioridade_efetiva`: calcula a prioridade que vale para uma tarefa neste instante. Sob herança, é o maior valor entre o detentor do recurso e o das tarefas

suspensas esperando por ele; sob teto, é o valor pré-calculado do recurso; sem protocolo, ou fora da seção crítica, é a prioridade base da tarefa.

- Parâmetros:
 - p: tarefa a avaliar;
 - tarefas: conjunto de tarefas usado para localizar quem está suspenso esperando por ela, sob herança;
 - protocolo: protocolo de correção ativo;
 - tetos: mapa de recurso para o teto calculado por `calcular_tetos`;
- Retorna:
 - o valor de prioridade a ser usado na seleção deste instante;
- `classificar_suspensoes`: depois da simulação, examina os períodos de suspensão registrados em cada tarefa e classifica cada um como bloqueio direto ou inversão de prioridades.
 - Parâmetros:
 - `processos`: lista de tarefas já simuladas;
 - Retorna:
 - uma lista com um registro por período de suspensão, contendo a tarefa, o recurso, o início e o fim da suspensão e a classificação;

`control/politicas.py`

- `Politica` (classe): declara como um algoritmo escolhe a próxima tarefa, sem descrever o mecanismo em si.
 - Atributos:
 - `algoritmo`: identificador do algoritmo;
 - `chave_selecao`: função de comparação usada para escolher a próxima tarefa;
 - `preemptivo`: se a tarefa roda por uma unidade de tempo ou até concluir;
 - `usa_quantum`, `suporta_envelhecimento`, `suporta_recurso`: indicam qual laço do motor deve tratar esse algoritmo;

- `POLITICAS`: mapa de cada algoritmo para sua `Politica` correspondente. É o que `simular_escalonamento` consulta para saber qual comportamento aplicar.
- `chave_prioridade_com_envelhecimento`: fábrica que devolve a chave de seleção usada pela Prioridade Cooperativa quando o envelhecimento está ativo. A prioridade efetiva cresce com o tempo desde o último despacho da tarefa (ou desde o ingresso, se ela nunca rodou) e volta ao valor base assim que ela é despachada de novo.
 - Parâmetros:
 - `alfa`: fator de crescimento da prioridade por unidade de tempo em espera;
 - Retorna:
 - uma função de chave (tarefa, instante atual) → critério de comparação, pronta para ser usada em `selecionar_proximo`;

`control/gerador.py`

- `sortear_tarefas`: gera um conjunto de tarefas aleatórias.
 - Parâmetros:
 - `n`: quantidade de tarefas a gerar;
 - `faixas`: intervalos a usar (usa os padrões de `Faixas` se omitido);
 - Retorna:
 - lista de tarefas sorteadas;
- `rodar_lote`: sorteia vários cenários e roda os seis algoritmos em cada um, para comparar médias.
 - Parâmetros:
 - `n_cenarios`: quantidade de cenários a sortear;
 - `n_tarefas`: quantidade de tarefas por cenário;
 - `faixas`: intervalos do sorteio;
 - `ctx_time`: tempo de troca de contexto;
 - `quantum`: quantum usado quando o Round Robin entra no lote;
 - Retorna:
 - mapa de cada algoritmo para suas médias de T , T_w e tempo até a primeira execução;

control/persistencia.py

- `salvar_cenario`: grava um conjunto de tarefas em arquivo.
 - Parâmetros:
 - `tarefas`: lista de tarefas a salvar;
 - `caminho`: arquivo de destino;
- `carregar_cenario`: lê um conjunto de tarefas de um arquivo salvo anteriormente.
 - Parâmetros:
 - `caminho`: arquivo de origem;
 - Retorna:
 - lista de tarefas reconstruída a partir do arquivo;

model/processo.py

- `Processo` (classe): representa uma tarefa e acumula, ao longo da simulação, tudo o que aconteceu com ela.
- `adicionar_processamento`: registra um período de execução.
 - Parâmetros: `inicio`, `fim` do período;
 - Retorna: a duração efetivamente executada (trunca se ultrapassar o tempo restante);
- `adicionar_troca_contexto`: registra um período de troca de contexto.
 - Parâmetros: `inicio`, `fim` do período;
 - Retorna: a duração do período;
- `get_turnaround`: calcula o tempo de execução total da tarefa (T).
 - Retorna: o instante de término da última execução menos o instante de chegada;
- `get_espera`: calcula o tempo de espera da tarefa (T_w).
 - Retorna: `get_turnaround()` menos `duracao`;
- `get_tempo_ate_primeira_execucao`: calcula quanto tempo a tarefa esperou até rodar pela primeira vez.
 - Retorna: o instante de início da primeira execução menos o instante de chegada;

- `verificar_estado`: consulta o que a tarefa estava fazendo em um instante específico.
 - Parâmetros: `instante` a consultar;
 - Retorna: o tipo do período que contém aquele instante, ou um estado descritivo (antes da chegada, após a conclusão, etc.);

6 PARÂMETROS CONFIGURÁVEIS

A tabela 3 mostra os parâmetros utilizados pela simulação que podem ser configurados pelo usuário.

Parâmetro	Faixa válida	Padrão	Contexto aplicado
t_q (quantum)	Inteiro maior que 0 e maior que t_{tc}	2	Round-Robin
t_{tc} (troca de contexto)	Inteiro maior ou igual a 0	0	todos
α (envelhecimento)	Inteiro (0 desabilita)	0	PRIOc
Protocolo	Nenhum; Teto de prioridade; Herança de prioridade	Nenhum	PRIOp

Tabela 3 - parâmetros de entrada

7 FUNCIONAMENTO INTERNO

O funcionamento interno da simulação pode ser descrito pela seguinte sequência de eventos:

1. O simulador reúne as tarefas prontas no instante presente; se não houver nenhuma, ele avança o tempo para o próximo ingresso.
2. Sob PRIOp com recursos:
 - a. Entre as tarefas já chegadas, verifica quais deveriam estar suspensas agora: uma tarefa suspende se o próximo instante de execução dela cair dentro da própria seção crítica e o recurso já estiver em uso por outra tarefa. Ao identificar uma suspensão nova, guarda o instante em que ela começou; ao identificar que uma suspensão em aberto acabou (o recurso foi liberado), fecha o intervalo e grava um período "Suspensa" na tarefa, do início ao fim da espera.
 - b. As tarefas suspensas saem do conjunto de candidatas deste evento.
3. A política aplica a chave de seleção para escolher a próxima tarefa, considerando também o critério de desempate (menor ingresso e, depois, menor identificador). Com envelhecimento em PRIOc, a prioridade efetiva usada na chave é sempre recalculada, não armazenada.
 - Em PRIOp, entre as candidatas restantes, escolhe pela prioridade efetiva: quem não detém nenhum recurso agora usa a prioridade base; sob herança, quem detém um recurso e tem alguém suspenso esperando por ele assume o maior valor de prioridade entre eles; sob teto, quem detém um recurso assume o teto daquele recurso, calculado uma única vez no início da simulação, independente de haver disputa real; sem nenhum protocolo, o detentor mantém a prioridade base mesmo segurando o recurso (o que pode levar a inversão de prioridade).
4. O simulador realiza a troca de contexto, se entrou uma tarefa diferente ou se é a primeira despachada. É adicionado um período "CTX" à tarefa, com a duração do custo.
5. Se houver preempção, a tarefa é executada em uma unidade de tempo; se não, é executada até o fim. Em Round-Robin, é executada em um quantum (subtraída a troca de contexto). É registrado um período de "Execução" correspondente na tarefa.
6. Se a tarefa foi concluída, seus tempos são somados aos totais e ela é removida do conjunto de pendentes. Se não, ela permanece entre as pendentes; sob Round-Robin, ela vai para o fim da fila.
7. Repete-se até não sobrar nenhuma tarefa.

8 CONVENÇÕES DE SIMULAÇÃO

A simulação segue as seguintes convenções:

- A progressão do tempo se dá em unidades inteiras discretas.
- Em caso de empate de critérios de escalonamento entre tarefas, prioriza-se a de menor identificador.
- A troca de contexto é computada sempre que há a troca de uma tarefa por outra diferente ou no momento em que a primeira tarefa é despachada.
- Sob Round-Robin, o custo de troca de contexto é descontado do tempo concedido à tarefa. Desse modo, por exemplo, uma troca de custo 1 com quantum igual a 3 deixará apenas 2 unidades de tempo para a execução da tarefa. Por isso, o quantum deve ser maior que o custo da troca
- Em Round-Robin, a tarefa preemptada ao fim do quantum tendo processamento restante vai para o fim da fila de tarefas prontas.
- A escala de prioridades tem sentido crescente: quanto maior for o número de prioridade de uma tarefa, mais prioritária ela é.
- O início e duração do uso de um recurso por uma tarefa são relativos ao tempo de processamento da tarefa, não ao tempo da simulação.
- A geração de tarefas para simulação normal e por lotes utiliza, por padrão, as faixas:
 - Ingresso: 0-8;
 - Duração: 1-6;
 - Prioridade: 1-5;

9 DEPENDÊNCIAS E AMBIENTE

No Windows, para rodar basta executar o arquivo `Simulador.exe`. Para rodar o programa a partir do script, a versão do Python requerida é 3.10 ou superior. Não são utilizadas bibliotecas externas. Utiliza-se apenas as bibliotecas nativas:

- `tkinter` – GUI de desktop (nativa no Windows; no Linux e MacOS pode ser necessário instalar o pacote `python-tk` pelo gerenciador de pacotes do sistema (`apt`, `dnf`, `brew` etc.);
- `json` – exportação e importação de cenários;
- `random` – gerador de números aleatórios;
- `dataclasses` – modelagem de classes;
- `copy` – fazer cópias de objetos;
- `enum` – modelagem de enumeradores;

O executável foi criado utilizando o Pyinstaller, por meio do comando:

```
pyinstaller -Fw main.py -n Simulador.exe
```